

中正大學編譯器專案

KaiyoHugo

Contents

專案檔案	2
環境要求	3
1. 手動安裝	3
2. Flake	3
編譯及運行	3
編譯 Grammar file	3
編譯 Runtime	3
編譯 C	3
架構	4
特殊點	4
Variable Shadowing	4
語法	5
Dynamic Allocation	6
型別系統	7
Control Flow	8
設計思路與解釋	9
new 和 Link List 和 Functional Language	9
Type System 與 Lambda Calculus	9
變數中不能放 Function	10
範例介紹	11
CTOR	11
Mandelbrot Set	11
Snake	12
2048	13

專案檔案

```
.
├── README.pdf # Readme
├── CFood.g4 # Grammar File
├── flake.nix # Nix 環境設置檔，環境會設置的完全和我一樣
├── flake.lock
├── justfile
├── Makefile
├── antlr.jar
├── Cargo.lock
├── Cargo.toml
├── CFood.g4
├── LICENSE
├── output.ll
├── output.o
├── output.out
├── src
│   ├── antlr
│   │   └── mod.rs # 和一些 ANTLR 生成的程式碼
│   ├── checker # Type Checker
│   ├── compiler # LLVM Codegen 相關的程式碼
│   ├── cst # Concrete Syntax Tree
│   ├── error.rs
│   ├── lib.rs
│   ├── main.rs
│   └── print.rs
├── swl # SoftWare Library
│   ├── Cargo.toml
│   └── src
│       └── lib.rs
├── target
│   └── release
│       └── libswl.so # 這個編譯後出現
└── tests # 測試的檔案
    ├── 1.c
    ├── 2.c
    └── 3.c
```

環境要求

我使用 Rust 和 LLVM 18 來編寫這次的作業，因此運行會需要 Rust。

有兩個方法可以安裝

1. 手動安裝

依照 <https://rust-lang.org/learn/get-started/> 上的方法安裝 Rust。

除了 rust 外還需要 `llvm_18`, `libffi`, `libxml2` 這三個 Library 和用於 linking 的 gcc，如果助教沒辦法安裝 llvm 18 話可以嘗試更改 `Cargo.toml` 中的版本

詳細列表看這裡：<https://docs.rs/crate/inkwell/0.9.0/features>

```
[dependencies.inkwell]
version = "0.9.0"
- features = ["llvm18-1"]
+ features = ["llvm19-1"]
```

但我不保證版本不同能正常編譯，尤其是 ≤ 14 的版本，所以推薦助教使用 Flake 保證環境正確。

2. Flake

必須要有 <https://nixos.org/> 安裝且啟用 Flake experiment。

然後運行 `nix develop` 就會自動安裝了。

編譯及運行

編譯 Grammar file

```
make build-grammar
```

編譯 Runtime

這個要先執行，會生成 `target/release/libswl.so`

```
make build-swl
```

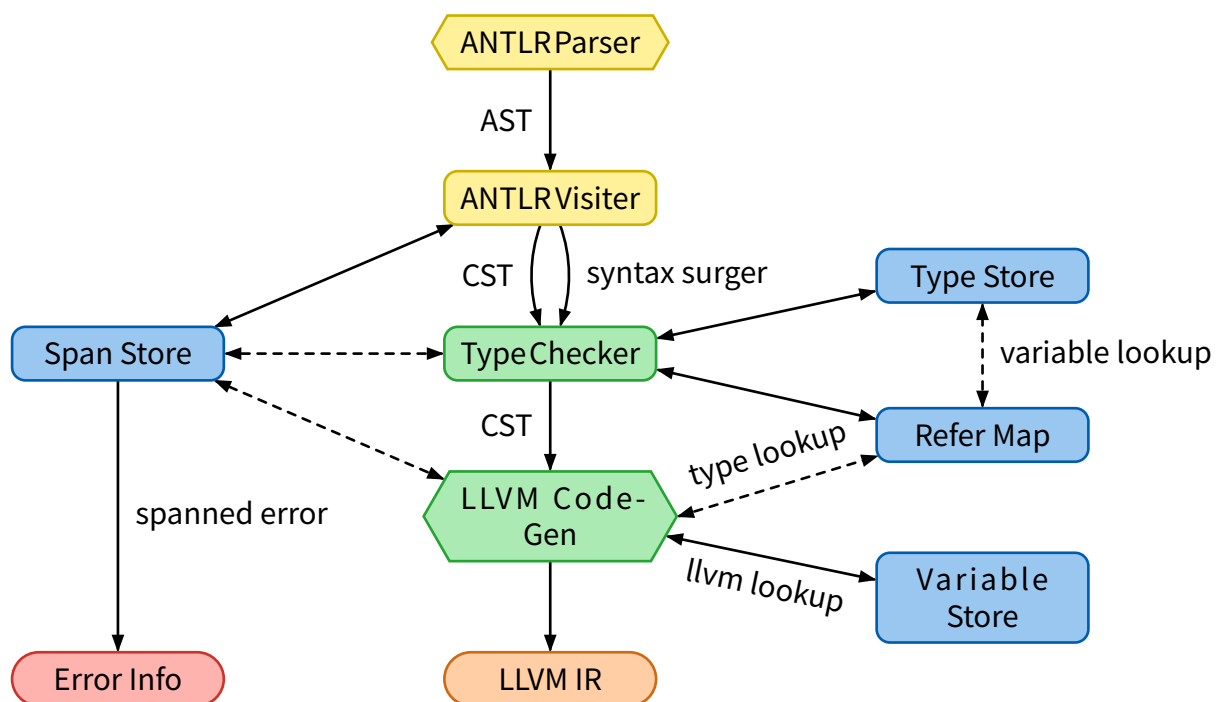
編譯 C

```
make run FILE=./tests/1.c
```

如果要觀察 `cst` 和型別訊息可以用 `make astuin` 觀察

`astuin` 要另外安裝：<https://github.com/KAIYOHUGO/Astuin>

架構



一開始 ANTLR 產生 AST，然後我將他重新生成 Concrete Syntax Tree，包含像是將許多語法轉成相等的形式 (Syntax Surger)，來減少 CodeGen 的複雜度，例如 For 迴圈會轉成 While 迴圈，Apply List 會變成 ExprCall 等等。每個 CST Node 上都會有一個 Unique ID，用來標記對應程式碼的 Span 和其他資訊，像是 Type Info 或 LLVM IR。

接著生成的 CST 會經過 Type Check，這個過程會生成 Type Store 和 Refer Map，Type Store 儲存所有 Type，而 Refer Map 用於標記變數對應的宣告 Tree Node，所以 CodeGen 時是靠它來確定變數查詢，這樣就不用重複實作變數尋找的邏輯。

最後 CodeGen 會使用 Variable Store 來存放已經生成的變數和函數，用 Type Store 和 Refer Map 來查詢型別和變數。

而 Span Store 用於生成可以標記錯誤範圍的錯誤輸出，讓編譯器的錯誤訊息更精確，主要用於 Type Check 和 CST 輸出。

特殊點

Variable Shadowing

變數的宣告我有實作 Variable Shadowing，是基於 C++ 的設計，每一個 Block 都可以重新宣告變數，範例如下。

```
int a = 10;
{
    // a = 10
    int a = 2;
    // a = 2
    int a = 3; // Error, duplicate declare
}
// a = 10
```

語法

我新增了 3 種新的 Token。

```
KW_type: 'type';
TYPE: [A-Z] [a-zA-Z0-9_]*;

KW_let: 'let';
```

第一個 type 是用於 type alias，也就是只是幫 = 右邊的 type 起名字，和直接用沒有差別，然後我讓大寫開頭的 Ident 變成 type alias 使用。

```
type Complex = (float, float);
```

第二個 let 是用來自動推導 type 的，也就是 `int a = 1` 和 `let a = 1` 是一樣的。

除了 Token 外，我還定義了類似 Tuple 的 Type —— (A, B, C...)，我稱他為 Apply List，Apply List 除了完全符合原本的 C 語法，還提供了非常簡單的 Data Type 的功能。

```
// The same
printf("Hello, World, %d", 10);
printf "Hello, World, %d" 10;
```

在 Grammar 上我新增了與上方對應的 Rule

```
// 給 type 用
ty_decl: KW_type TYPE ASSIGN PAREN_L tys PAREN_R SEMICOLON;

// 給 let 用
let_expr: KW_let var ASSIGN expr;

// 給 type 用
ty_kind
: TY_int    # ty_kind_ty
| TY_float  # ty_kind_ty
| TY_str    # ty_kind_ty
| TY_void   # ty_kind_ty
| TY_bool   # ty_kind_ty
| TYPE      # ty_kind_type
;

// 給 apply list 用
apply_list
: PAREN_L PAREN_R
| PAREN_L args PAREN_R;

args
: expr COMMA args
| expr
|;
```

我有支援在變數呼叫時同時賦值的功能，也就是 `int a = 1` 這種語法。

```
var_decl: var_decl_ty var_decl_init SEMICOLON;
var_decl_ty: ty IDENT | ty IDENT BRACKET_L NUMBER BRACKET_R;
var_decl_init: ASSIGN expr | ;
```

因為 Type-Widening 容易讓人感到困惑，所以現代程式語言都用轉成手動轉換，而我採用 Rust 的語法，也就是 `value as Type`。

```
expr_cast
: expr_unary # expr_cast_pass
| lhs=expr_unary KW_as rhs=ty_kind # expr_cast_use
| lhs=expr_unary KW_as REFER rhs=ty_kind # expr_cast_refer_use
;
```

我也增加了 Pointer Casting，也就是 `value as &Type`，和對應的取址操作 `&value`。

```
refer
: REFER IDENT;
```

我有以 Syntax Surger 的方式加入 for 迴圈

```
for_stmt
: KW_for PAREN_L init=inline_stmts SEMICOLON cond=expr SEMICOLON
mutate=inline_stmts PAREN_R stmt;
```

for 迴圈會被轉成對應的 While 迴圈處理。

我有新增 `&&` 和 `||` 對 `bool` 型別做邏輯運算

```
: expr_cmp # expr_logic_pass
| lhs=expr_cmp logic_preced_op rhs=expr_logic # expr_logic_use
;
```

Dynamic Allocation

```
type A = (int, int, int);

int main() {
    let a = new 10 20 30;
    printf("%d %d %d", a as &A);
    delete a;
    return 0;
}
```

`new` 會將所有輸入值存到 heap 上，並且回傳 Pointer，這個語言的 Pointer 是 untyped，也就是 `int`，`delete` 會 free pointer，所以 Pointer 是 immutable，想要變更值只能重新創建，然而這樣就足夠用 Functional Language 的方式創建 LinkList 等 Type，詳細看 `snake.c` 和底下說明。

型別系統

這個 type system 是參考 Hindley–Milner type system 改成的，首先定義符號如下

粗體大寫英文字 $\mathbb{T}, \mathbb{K}, \dots$ 是 apply list，是由一堆 monotype 組成，像是 `int`, `float`。

ε 是屬於 $\mathbb{T}$ 的特殊型，也就是什麼都沒有的 apply list。

σ 則是一個函數，可以接受輸入。

$$\begin{aligned}\mathbb{T} &= (\tau_1, \tau_2, \tau_3, \dots) \\ \varepsilon &= () \\ \sigma &= \mathbb{T} \rightarrow \mathbb{Y} \mid \sigma \rightarrow \sigma\end{aligned}$$

我總共設計 4 種規則，且這個系統並沒有 polytype，且是 first order function 以減少複雜度。

介紹	Rule	Rule
基本的 type	$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \text{Var}$	
$\varepsilon \rightarrow x$ 就是 x	$\frac{\Gamma \vdash x : \sigma}{\Gamma \vdash x : \varepsilon \rightarrow \sigma} \text{I } \varepsilon$	$\frac{\Gamma \vdash x : \varepsilon \rightarrow \sigma}{\Gamma \vdash x : \sigma} \text{E } \varepsilon$
apply list 合併/分開	$\frac{\Gamma \vdash M : (\mathbb{T}, \mathbb{Y}) \rightarrow \sigma}{\Gamma \vdash M : \mathbb{T} \rightarrow \mathbb{Y} \rightarrow \sigma} \text{S } \rightarrow$	$\frac{\Gamma \vdash M : \mathbb{T} \rightarrow \mathbb{Y} \rightarrow \sigma}{\Gamma \vdash M : (\mathbb{T}, \mathbb{Y}) \rightarrow \sigma} \text{M } \rightarrow$
呼叫函數	$\frac{\Gamma \vdash M : \mathbb{K} \rightarrow \sigma \quad \Gamma \vdash N : \mathbb{X} \rightarrow \mathbb{K}}{\Gamma \vdash MN : \mathbb{X} \rightarrow \sigma} \text{App}$	$\frac{\Gamma \vdash M : \mathbb{Y} \rightarrow \mathbb{T} \quad \Gamma \vdash N : \mathbb{X} \rightarrow (\mathbb{Y}, \mathbb{K})}{\Gamma \vdash MN : \mathbb{X} \rightarrow (\mathbb{T}, \mathbb{K})} \text{Ext}$

我拿實際範例來表達我的想法。

```

type V = (int, int);

V add(int a, int b, int c, int d) {
  return (a + c, b + d);
}

V add_3(V a, V b, V c) {
  retrun add add (a, b, c);
}

```

上方的 add_3 會將 3 個 Vec 相加在一起，我們知道 add add (a, b, c) 的 type 必須是 v，也就是 $\Gamma \vdash \text{add add } (a, b, c) : V$ ，這時候就可以使用上面的規則解出它的確是 Vec。

$$\frac{\frac{}{\Gamma \vdash \text{add} : (V, V) \rightarrow V} \text{Var} \quad \frac{\frac{}{\Gamma \vdash \text{add} : (V, V) \rightarrow V} \text{Var} \quad \Gamma \vdash (a, b, c) : (V, V, V)}{\Gamma \vdash \text{add add } (a, b, c) : (V, V)} \text{Ext}}{\Gamma \vdash \text{add add } (a, b, c) : V} \text{App}$$

如果仔細觀察上面的證明，可以看到 $(a, b, c) : (V, V, V)$ 我並沒有證明，所以我底下要來證明，可以看到其實他就是一直做 Application，而且是按照順序的，這也是我叫他 apply list 的原因，所以本質上 a b c 和 (a,b,c) 是一樣的，只差在運算子優先值。

$$\frac{\frac{\frac{}{\Gamma \vdash a : V} \text{Var}}{\Gamma \vdash a : \varepsilon \rightarrow V} \text{I } \varepsilon \quad \frac{\frac{\frac{}{\Gamma \vdash a : V} \text{Var} \quad \frac{}{\Gamma \vdash b : \varepsilon \rightarrow V} \text{I } \varepsilon}{\Gamma \vdash (b, c)} \text{Ext} \quad \frac{}{\Gamma \vdash c : V} \text{Var}}{\Gamma \vdash (a, b, c) : (V, V, V)} \text{Ext}$$

要特別強調 $(\mathbb{T}, \mathbb{K})$ 其實就是把兩個 apply list 鏈接在一起的意思，所以 $(\varepsilon, \mathbb{T})$ 就是 $\mathbb{T}$ 。

除此之外 curry function 或任何的 non-concrete type 都不能放在變數裡面，這個 curry function 僅限於在 expression 中存在。

Control Flow

我 if, if ... else ..., while 和 for 都有實作，且它們都可以多層包覆。Function Calling 我有實作 Hoisting，也就是 Function 宣告的順序無關，都可以找到 Function，也可以遞迴呼叫。而 Function 有實作 return，當沒有 return 則會預設為 0。

設計思路與解釋

這邊比較像是介紹為什麼這樣設計或為什麼有這樣的限制，如果助教已經理解就不需要看。

new 和 Link List 和 Functional Language

我們可以寫出一個簡單的 Link List

```
type Value = (int, int)

int node_value(int node) {
    return _node_value (node as &Snake_Node);
}

int _node_value(Value value, int next) {
    return value;
}

int node_next(int node) {
    return _node_next (node as &Snake_Node);
}

int _node_next(Value value, int next) {
    return next;
}

int make_node(Value value, int next) {
    return new value next;
}
```

當我們要創建可以像這樣：

```
let link_list =
    make_node 6 8
    (make_node 5 8
     (make_node 4 8 0));
```

如果要尋訪可以用遞迴的方式處理，這樣就可以用一種介於函數語言（Haskell）和 C 中間的方式來實作動態大小的結構。更複雜的範例可以看 `snake.c`

Type System 與 Lambda Calculus

這個 apply list 的 type system 是從 lambda calculus 來的，當我們定義 `type a = (int, int)` 時，在 lambda calculus 則是如下。

$$A := \lambda x. \lambda y. \lambda z. \lambda f. fxyz$$

也就是 lambda calculus 的 pair (structure) 的定義，其中的 f 是 getter，這個後面會解釋，而 xyz 就是會帶入 f 的參數，這個與我的 C 中的 (1,2,3) 或 1 2 3 對應，當我們要取值時會寫 getter 如下。

$$\text{Get1} = \lambda s.s(\lambda x.\lambda y.\lambda z.x)$$
$$\text{Get2} = \lambda s.s(\lambda x.\lambda y.\lambda z.y)$$
$$\text{Get3} = \lambda s.s(\lambda x.\lambda y.\lambda z.z)$$
$$S = A\ 1\ 2\ 3 = \lambda f.f\ 1\ 2\ 3$$
$$\text{Get1}\ S = \lambda f.f\ 1\ 2\ 3(\lambda x.\lambda y.\lambda z.x) = (\lambda x.\lambda y.\lambda z.x)\ 1\ 2\ 3 = 1$$
$$\text{Get2}\ S = \lambda f.f\ 1\ 2\ 3(\lambda x.\lambda y.\lambda z.y) = (\lambda x.\lambda y.\lambda z.y)\ 1\ 2\ 3 = 2$$
$$\text{Get3}\ S = \lambda f.f\ 1\ 2\ 3(\lambda x.\lambda y.\lambda z.z) = (\lambda x.\lambda y.\lambda z.z)\ 1\ 2\ 3 = 3$$

其中的 S 就是帶值的狀態，也可以看到能正常取值，但也可以看到其實 Get1 在做把帶進來的參數做拆解然後回傳而已，所以我設計的 C 則會如下。

```
int get_1(int x, int y, int z) {  
    return x  
}  
  
A s = 1 2 3;  
// first == 1  
int first = get_1 s;
```

所以可以看到其實我大量借用 lambda calculus 的想法，再將其轉換成更簡單的形式，當然除了這個我還多加了可以拆分 apply list 的功能（詳細看 型別系統）

變數中不能放 Function

```
int add(int a, int b) {  
    ...  
}  
  
// int -> int  
let add_1 = add 1;
```

這看起來是簡單不過的程式，但在實作時會有許多問題，我在這裡分析其優缺點。

如果要放 stack 則我們需要知道它的大小，但我們要先知道他所有可能的大小，最簡單的記憶體配置是 `ptr|arg1|arg2|...`，那我們要知道所有可能賦值給 `add_1` 的 curry 的數量，而這是相當複雜的，尤其在正常的 C 中，因為要分析什麼值會賦予給 `add_1` 就相當於要讓編譯器解 pointer 計算結果的解，當然最簡單的解就是直接取整個程式中參數最多有多少來解，但這個會導致生成大量不必要的 LLVM IR。在 Rust 中則是會規定只能是同一個 curry function，或者以 Rust 的講法：「A closure expression produces a closure value with a unique, anonymous type」，所以即使都是一樣參數的函數 $F_n(isize) \rightarrow isize$ 也不能重複賦值。如果在 Functional Language 中則不會有 pointer 的問題且可以簡單得知所有可能的大小。

如果要放 heap 則簡單多，只要做 vtable 就可以了，但這也導致使用者要自己釋放記憶體，而 Haskell 是以來 garbage collection 來處理。

範例介紹

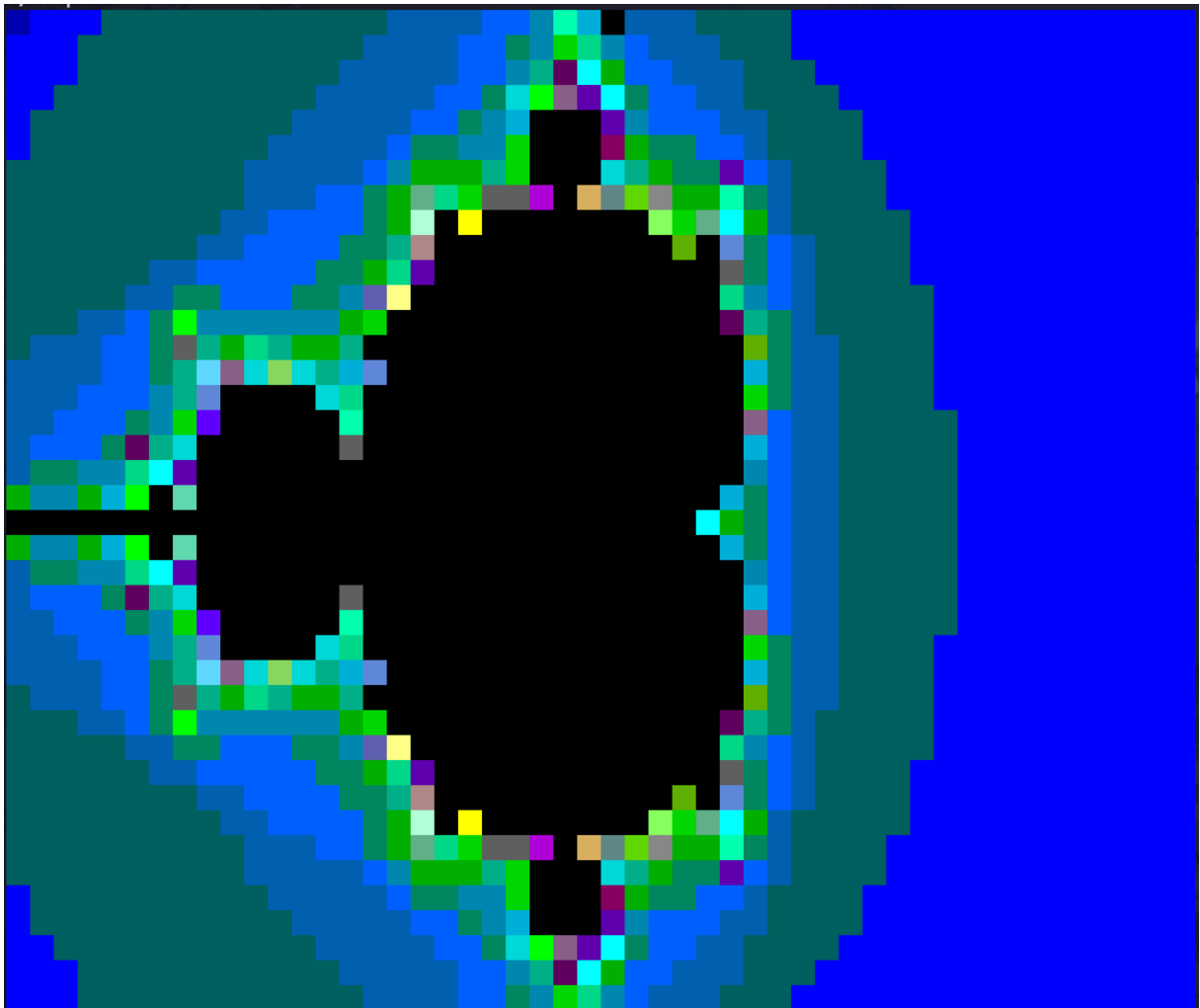
CTOR

```
float a = 2.0;  
float b = 3.0;  
// ctor magic  
float r = a ## b;  
int main(void) {  
    printf("a = %f, b = %f\n", a, b);  
    printf("a ## b = %f\n", r);  
}
```

Code 1: 5.c

在全域變數中我會使用 CTOR 來初始化函數，也就是全域變數不止可以放常數，也可以放需要計算的值，這些初始化是藉由 CTOR 來實作的，會在 main 之前完成。

Mandelbrot Set



程式看 mandelbrot.c，裡面將 apply list 作為 complex number 的 type 來使用，然後輸出彩色的 mandelbrot set。

```

type Complex = (float, float);

float real_part(float r, float _) { return r; }
float image_part(float _, float i) { return i; }

Complex complex_mul(Complex a, Complex b) {
    return (real_part a * real_part b - image_part a * image_part b,
            real_part a * image_part b + image_part a * real_part b);
}

Complex complex_add(Complex a, Complex b) {
    return (real_part a + real_part b, image_part a + image_part b);
}

float complex_abs_square(Complex a) {
    return real_part a * real_part a + image_part a * image_part a;
}

```

Snake

```

score: 36
wasd move, q quit
#####
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##          000000000000              ##
##          00          00            ##
##          @@          00            ##
##                                     000000000000 ##
##                                     00          ##
##                                     00          ##
##                                     00000000    ##
##                                     00          ##
##                                     00          ##
##          000000000000000000000000000000000000 ##
##                                     **          ##
#####

```

程式看 `snake.c`，一個非常簡單版本的貪吃蛇，用 `wasd` 移動，用來證明我的編譯器是能夠編譯複雜的檔案，且其中使用了 `new` 和 `delete` 來儲存動態長度蛇的座標，也使用 Runtime Library 所提供的 `enable/disable terminal raw mode` 來讓 `scanf` 能夠直接讀取使用者的輸入。

2048

```
2048    score: 1248
wasd move, r restart, q quit
linked-list board version

+-----+-----+-----+-----+
|  2  |  4  |  8  |  2  |
+-----+-----+-----+-----+
|  4  | 32  | 16  |  4  |
+-----+-----+-----+-----+
| 32  |  2  | 128 |  2  |
+-----+-----+-----+-----+
|  2  |  4  |  8  | 32  |
+-----+-----+-----+-----+
game over
```

一個經典的小遊戲，用 wasd 移動，裡面用 new/delete 來動態儲存盤面，也使用 Runtime Library 所提供的 enable/disable terminal raw mode 來讓 scanf 能夠直接讀取使用者的輸入。